

VECTA — 進捗レポート 2026-07-26

[概要](#)[仕組み](#)[検証](#)[リスクと残タスク](#)[用語集](#)

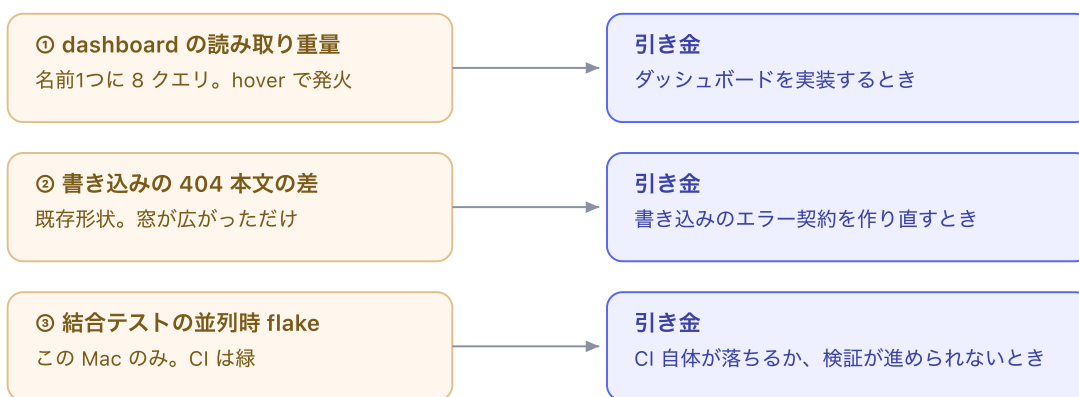
リスクと残タスク

Abstract

What 直さずに記録した3件と、それぞれを直すべき引き金。

Why 書き留めていない負債は複利で膨らむ。放置してあること自体は害ではない。

How 「いつ直すか」を条件で書いた。時期ではなく、事象で。



記録した3件。すべて HANDOFF.md の債務欄に、この引き金つきで書いてある。

Detail

1. ダッシュボードがワークスペース全体を読む

`/projects/:id/dashboard` は「ダッシュボードは Step 4 で実装します (プロジェクト名)」と出すだけのスタブですが、そのプロジェクト名のためにワークスペース一括読み取り (8クエリ) が走ります。変更前は3カラム1行の読み取りでした。往復回数は2回そのまま、**読む量だけが増えた**回帰です。

さらにナビゲーションタブには `prefetch="intent"` が付いているため

(`apps/web/app/shell/app-bar.tsx:231`)、ダッシュボードのタブに**マウスを載せた**だけでこ

の読み取りが飛びます。

直さなかった理由: 直すには、今回消した「プロジェクト行だけを読む専用のクエリ」を、スタブ画面のためだけに復活させることになります。実装後のダッシュボード（EVM 表示）はどのみちワークスペース相当を読むので、二重に無駄です。

引き金: ダッシュボードを実装するとき、その画面の読み取りを設計する時点で解消する。もし名前だけ表示のまま本番に出す判断になるなら、その時点で軽量クエリを戻す必要がある。

2. 削除済みプロジェクトへの保存が、ゲートと違う形の 404 を返す

メンバーだったプロジェクトの行が消えたあとに保存 POST を投げると――

	変更前	変更後
経路	<code>requireProjectAccess</code> の行読み取りが <code>null</code> → 404 を throw	<code>membership</code> を同期で読むので通過し、 <code>applyCommands</code> まで到達
応答	<code>data(null, { status: 404 })</code>	<code>{ ok:false, code:"NOT_FOUND" }</code> の 404

この応答形状は既存のものです。変更前も「行読み取りとトランザクションの間に削除が走る」レースで到達し得ました。今回は、その発生窓が「principal 読み取りから書き込みまで」に広がっただけです。

情報は漏れません。理由は3つあります。

- 観測できるのは**そのプロジェクトの（元）メンバー本人だけ**。部外者は最初のメンバーシップ照合で弾かれる
- `project_memberships` はプロジェクト行に `ON DELETE CASCADE` で紐づいているため、この状態は**1リクエスト内のレースでしか存在しない**。次のリクエストではメンバーでなくなり、ゲートの 404 に収束する
- 読み取り側の 404 は本文まで完全に一致したまま。外部から「存在するかどうか」を測る手段にはならない

直さなかった理由: `NOT_FOUND` はクライアント側の**型付きの契約**です（`apps/web/app/wbs/wbs-app.tsx:532`、`apps/web/app/masters/master-app.tsx:49`）。保存キューはこれを受けて楽観的更新を穏当に巻き戻します。不透明な 404 を throw に変えると、この巻き戻しがエラー画面に変わります。性能改善の副作用として書き込みパスのエラー契約を作り変えるべきではない、と判断しました。

引き金: 書き込みパスのエラー契約を作り直すとき、または監査が「読み書き両方でバイト単位に同一の 404」を要求したとき。閉じるのは1行（`runCommandAction` の `NOT_FOUND` を throw に寄せる）だが、クライアント側の分岐も同時に直す必要がある。

3. 結合テストが並列実行で落ちる（この Mac のみ）

詳細は[検証](#)に書きました。要点だけ再掲します。

- 8つのテストファイルがそれぞれ Postgres コンテナを起動する。セットアップは60秒猶予、テスト本体は vitest 既定の5秒のまま
- 直列なら 46/46 緑、CI (ubuntu-latest) も緑。**コードの欠陥ではない**
- ただしローカルの `pnpm check` が素直に緑にならず、検証の手触りが悪い

引き金: CI 自体がこれで落ちたとき、または他の方法で完了できない検証をこれが塞いだとき。直し方はコンテナを使うファイルの `testTimeout` を上げるか、このパッケージの `maxWorkers` を絞るか。

4. 次にやること

順	項目	状態
1	コマンドコア経由の LLM 操作 (ADR 0012 の vision 機能)	要件未取得。ユーザーへの質問あり — 何ができるべきか (読むだけの Q&A か、変更案の提示か、適用まで行うか)、変更を誰が承認するか、ブラウザで動かすか <code>/mcp</code> 経由か
2	クライアント/サーバー境界を構造で強制する	今回の変更で、ミドルウェアが persistence を直接 import するようになった。成果物走査と import 制限の2段構えを作る
3	定期アーキテクチャレビュー	未着手
4	セキュリティレビュー (サプライチェーン・CI)	未着手

参考文献

1. PostgreSQL — Foreign key ON DELETE CASCADE — postgresql.org/docs/current/ddl-constraints
2. リポジトリ内: `docs/agents/HANDOFF.md` (債務欄。本レポートの3件はすべてここに引き金つきで記録済み)
3. リポジトリ内: `packages/persistence/src/schema.ts` (`project_memberships` の CASCADE 定義)